

# Uncertainty-Calibrated Diabetic Retinopathy Grading with Counterfactual Visual Explanations

Implementation, Coding Quality and Functionality

Gayathri · Cohort E02 · Department of Computer Science and Engineering · 18 September 2026 · repository commit `ec6e161`

## 1. Implementation status

All six layers of the design are implemented as an installable package with a console script, built in six reviewed phases (Table 3). Each phase was merged only when linting, formatting, static typing and the full test-suite passed. The repository currently contains about 6 000 lines of Python and YAML, of which roughly 1 100 are tests.

7

packages · 35 modules

59

tests passing (50 functions)

0

ruff / black / mypy findings

22

config compositions checked

Table 1 — Code size by component (Python + YAML, excluding docs).

| component            | files | lines | contents                                                                         |
|----------------------|-------|-------|----------------------------------------------------------------------------------|
| dr_uq/data           | 6     | 948   | loaders, splits, preprocessing, quality, datamodule                              |
| dr_uq/models         | 3     | 266   | backbones, GradingModel, LitGrader                                               |
| dr_uq/uncertainty    | 7     | 445   | scores, protocol, temperature scaling, MC dropout, ensemble, factory             |
| dr_uq/selective      | 4     | 290   | gate, risk-coverage, stratification                                              |
| dr_uq/counterfactual | 5     | 696   | generator, invert, optimise, validate                                            |
| dr_uq/evaluation     | 5     | 694   | calibration, grading, runner, report                                             |
| dr_uq/deploy         | 5     | 691   | ONNX export, TensorRT, profiler, CLI                                             |
| scripts              | 8     | 694   | train, evaluate, sweep, explain, train_generator, corpus, manifest, config check |
| configs              | 23    | 411   | Hydra groups and experiment presets                                              |
| tests                | 10    | 1124  | unit + integration                                                               |

## 2. Repository layout

```
dr_uq/  
  data/          loaders.py · splits.py · preprocess.py · quality.py · datamodule.py  
  models/        backbones.py · grading_model.py  
  uncertainty/    scores.py · base.py · temp_scaling.py · mc_dropout.py · ensemble.py ·  
  factory.py
```

```

selective/      gate.py · risk_coverage.py · stratify.py
counterfactual/ generator.py · invert.py · optimise.py · validate.py
evaluation/     calibration.py · grading.py · runner.py · report.py
deploy/        export_onnx.py · build_trt.py · profile.py · cli.py
configs/       data/ · model/ · uq/ · train/ · deploy/ · experiment/ · config.yaml
tests/         unit/ · integration/
scripts/       train.py · evaluate.py · sweep.py · explain.py · train_generator.py
               make_synthetic_corpus.py · build_manifest.py · check_configs.py
docs/          DECISIONS.md · MODEL_CARD_TEMPLATE.md · prc2/
dvc.yaml · pyproject.toml · uv.lock · .pre-commit-config.yaml · .github/workflows/ci.yml ·
Dockerfile · README.md

```

## 3. Coding quality

### 3.1 Conventions

- **Typed, documented public API.** Every public function and class carries type hints and a Google-style docstring; `mypy` passes on the whole package with `disallow_untyped_defs`.
- **No hard-coded choices.** Paths, hyper-parameters and method selections come from Hydra groups; `struct` mode rejects typos in overrides.
- **Single implementation of shared maths.** Uncertainty scores, ECE and QWK are implemented once and reused by training metrics, evaluation, gating and bootstrapping.
- **Guarded optional dependencies.** `TensorRT`, `LPIPS`, `pytorch-fid` and `StyleGAN2` code paths degrade with explicit messages instead of import failures.
- **Decision log.** Every non-obvious choice (24 entries so far) is recorded in `docs/DECISIONS.md` with its rationale.

### 3.2 Representative code

`dr_uq/models/grading_model.py`

```

class GradingModel(nn.Module):
    """Five-class DR grader: timm backbone → dropout → linear head.

    Args:
        backbone: A timm model created with ``num_classes=0`` (returns pooled features).
        num_features: Feature dimension of the backbone output.
        num_classes: Number of output grades (5).
        dropout: Dropout probability before the head (used by MC dropout).

    Attributes:
        dropout_active: When True, dropout is applied even in ``eval()`` mode so repeated
            forward passes are stochastic (MC dropout). Defaults to False.
    """

    def __init__(
        self, backbone: nn.Module, num_features: int, num_classes: int = 5, dropout: float = 0.2
    ) -> None:
        super().__init__()
        self.backbone = backbone
        self.head = nn.Linear(num_features, num_classes)
        self.p_dropout = float(dropout)
        self.num_classes = num_classes
        self.dropout_active: bool = False

    def features(self, x: Tensor) -> Tensor:
        """Pooled backbone features ``(B, F)``."""
        return self.backbone(x)

```

```

def forward(self, x: Tensor) -> Tensor:
    """Logits ``(B, 5)`` for a normalised image batch ``(B, 3, H, W)``."""
    f = self.features(x)
    f = F.dropout(f, p=self.p_dropout, training=self.training or self.dropout_active)
    return self.head(f)

def head_parameters(self) -> list[nn.Parameter]:
    """Parameters of the classifier head."""
    return list(self.head.parameters())

def backbone_parameters(self) -> list[nn.Parameter]:
    """Parameters of the backbone."""
    return list(self.backbone.parameters())

```

dr\_uq/selective/gate.py

```

class SelectiveGate:
    """Accept a prediction iff its uncertainty is strictly below ``tau``.

    Args:
        tau: Uncertainty threshold. ``tau=0`` refers everything (no uncertainty is < 0);
            ``tau=inf`` refers nothing.
        score: Which uncertainty to threshold. ``maxp``/``entropy`` are recomputed from the
            probabilities; ``mi`` uses the uncertainty vector supplied by the UQ wrapper.
    """

    def __init__(self, tau: float, score: Literal["maxp", "entropy", "mi"] = "entropy") -> None:
        self.tau = float(tau)
        self.score = score

    def uncertainty(self, probs: Tensor, u: Tensor) -> Tensor:
        """Effective uncertainty used for gating."""
        if self.score == "mi":
            return u
        return score_from_probs(probs, self.score)

    def __call__(self, probs: Tensor, u: Tensor) -> Tensor:
        """Return grades ``0..4`` for accepted samples and ``-1`` for referred ones."""
        eff = self.uncertainty(probs, u)
        grade = probs.argmax(dim=-1)
        refer = torch.full_like(grade, REFER)
        return torch.where(eff < self.tau, grade, refer)

    def accept_mask(self, probs: Tensor, u: Tensor) -> Tensor:
        """Boolean mask of accepted samples."""
        return self.uncertainty(probs, u) < self.tau

```

dr\_uq/selective/risk\_coverage.py

```

def exact_auroc(u: NDArray[np.float64], correct: NDArray[np.bool_]) -> float:
    """AUROC from the exact ordering: mean over ``i`` of error rate among the ``i`` most certain.

    Ties in ``u`` are broken pessimistically (errors first) so the value does not depend on
    the input order.
    """
    u = np.asarray(u, dtype=np.float64)
    correct = np.asarray(correct, dtype=bool)
    n = len(u)
    if n == 0:
        return float("nan")
    order = np.lexsort((correct.astype(int), u)) # sort by u, then errors (0) before correct
    (1)

```

```

errors = np.cumsum(~correct[order])
risks = errors / np.arange(1, n + 1)
return float(risks.mean())

```

```

def optimal_aurc(correct: NDArray[np.bool_]) -> float:
    """Theoretical minimum AURC for a perfect scorer with the given accuracy.

    With ``k`` correct out of ``n``, the perfect ordering accepts all correct samples first, so
    risk is 0 for ``i <= k`` and ``(i - k) / i`` afterwards.
    """
    correct = np.asarray(correct, dtype=bool)
    n = len(correct)
    k = int(correct.sum())
    if n == 0:
        return float("nan")
    i = np.arange(k + 1, n + 1)
    return float(((i - k) / i).sum() / n)

```

dr\_uq/counterfactual/optimise.py

```

def counterfactual_objective(
    gen_img: Tensor,
    x01: Tensor,
    target: Tensor,
    model: nn.Module,
    temperature: float,
    lambda_l1: float,
    lambda_lpips: float,
    perceptual: PerceptualLoss | None,
) -> tuple[Tensor, Tensor]:
    """`CE(f(G(w))/T, g') + λ1 ||G(w) - x||_1 + λ2 LPIPS(G(w), x)`; returns ``(loss,
    logits)``."""
    logits = model(to_grader_input(gen_img)) / temperature
    ce = F.cross_entropy(logits, target, reduction="none")
    l1 = (gen_img - x01).abs().mean(dim=(1, 2, 3))
    loss = ce + lambda_l1 * l1
    if perceptual is not None and lambda_lpips > 0:
        loss = loss + lambda_lpips * perceptual(gen_img, x01)
    return loss.sum(), logits

```

## 4. Testing

Tests are split into fast unit tests on a 40-image, 64-pixel synthetic fixture and integration tests that train, evaluate and explain end-to-end. Contracts from the design document are asserted directly (Table 2). The whole suite runs in about 30 s on CPU.

**Table 2 — Test-suite coverage by module.**

| test module                | what is verified                                                                                                                                                                                                                                                                                                                |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| test_data.py (12)          | No patient in more than one split; manifests byte-identical for a fixed seed and different across seeds; preprocessing output (512,512,3) uint8 and (3,512,512) float32; quality $\in [0,1]$ and blur lowers it; cache round-trip; class-balanced sampler $\approx 50/50$ and seeded; datamodule end-to-end incl. external mode |
| test_models.py (10 items)  | Every backbone returns (B,5); dropout_active=True changes outputs between passes and False does not; 512 <sup>2</sup> contract; TemperatureScaled and referable probability                                                                                                                                                     |
| test_uncertainty.py (11)   | ECE/MCE/NLL/Brier equal hand-computed values; ECE matches netcal to 1e-6; temperature fit lowers validation NLL and preserves argmax; MC-dropout and ensemble probabilities sum to 1 and are reproducible; all wrappers satisfy UQWrapper; factory errors                                                                       |
| test_selective.py (6)      | Gate refers everything at $\tau=0$ and nothing at $\tau=\infty$ ; risk-coverage coverage monotone; AURC of a perfect scorer equals the theoretical optimum; QWK equals scikit-learn; stratification totals; report aggregation with CIs on synthetic run dirs                                                                   |
| test_counterfactual.py (6) | Optimiser and inversion decrease their objectives; flip check agrees with the classifier; lesion overlap = 1.0 when $\Delta$ equals the mask and $\approx$ random baseline for random $\Delta$ ; Grad-CAM shape/range; Likert template                                                                                          |
| test_deploy.py (5)         | ONNX output equals PyTorch within 1e-4 on a fixed batch; CLI end-to-end with ONNX, checkpoint and MC-dropout back-ends; profiler; TensorRT guard                                                                                                                                                                                |
| integration (6)            | 2-epoch training writes best/last checkpoints and run metadata; evaluate.py produces every artefact for none/temp_scaling/mc_dropout/ensemble and in external mode; explain.py end-to-end                                                                                                                                       |

```
$ uv run ruff check . && uv run black --check . && uv run mypy && uv run python
scripts/check_configs.py && uv run pytest
All checks passed!
Success: no issues found in 37 source files
checked 22 compositions, 0 failures
59 passed in 26.22s
```

## 5. Functionality demonstration

The definition-of-done chain runs end-to-end on a CPU-only machine without any real data and was executed from a clean checkout:

```
pip install -e .[dev]
python scripts/make_synthetic_corpus.py          # 300 synthetic 5122 fundus images
+ masks + sample.png
pytest                                           # 59 passed
python scripts/train.py data=synthetic model=resnet50 train.max_epochs=1
python scripts/evaluate.py experiment=smoke      # report.json, reliability.png,
```

```
risk_coverage.{csv,png}, model.onnx, model_card.md
python scripts/explain.py data=synthetic explain.n_images=4 # counterfactuals, delta maps,
Grad-CAM, lesion consistency
dr-uq grade --engine runs/smoke/model.onnx --temperature 1.0 --tau 0.5 sample.png
```

## 5.1 Artefacts produced per evaluation

**Table 3 — Files written by scripts/evaluate.py for every run.**

| file                                                            | content                                                                                                                           |
|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| report.json                                                     | grading, calibration, selective-prediction metrics, reliability bins, operating point, referral budgets, stratification, git hash |
| predictions.npz / predictions_val.npz                           | per-image probabilities, uncertainty, labels, quality — consumed by the bootstrap aggregator                                      |
| reliability.png                                                 | reliability diagram with per-bin counts                                                                                           |
| risk_coverage.png,<br>risk_coverage_{test,val}.csv              | risk-coverage curves                                                                                                              |
| referral_budget.csv                                             | test coverage/error/QWK at validation-chosen thresholds for 5–50 % referral                                                       |
| stratification.csv                                              | referral rate and accuracy per quality bin and per true grade                                                                     |
| model_card.md                                                   | filled model card (intended use, metrics, limitations)                                                                            |
| model.onnx                                                      | backbone + temperature graph, verified against PyTorch                                                                            |
| config_resolved.yaml, run_meta.json,<br>requirements_freeze.txt | reproducibility metadata                                                                                                          |

## 5.2 Development history

**Table 4 — Commit history (one commit per reviewed phase).**

| commit  | phase     | content                                                                                             |
|---------|-----------|-----------------------------------------------------------------------------------------------------|
| 429f847 | Phase 0/1 | scaffolding, tooling, CI, data layer (loaders, splits, preprocessing, datamodule), synthetic corpus |
| e943e40 | Phase 2   | timm backbones, GradingModel, Lightning training recipe, multirun                                   |
| 8695498 | Phase 3   | uncertainty wrappers, scores, calibration metrics (netcal cross-check)                              |
| eece370 | Phase 4   | selective gate, risk-coverage, evaluate.py, bootstrap report aggregation, sweep.py                  |
| 4a69c5b | Phase 5   | generator interface, inversion, counterfactual optimisation, validation, explain.py                 |
| 8e90b2a | Phase 6   | ONNX export, TensorRT builder, profiler, CLI, Dockerfile, README, decisions                         |
| ec6e161 | Finalise  | definition-of-done chain verified from a clean state                                                |

## 6. Known gaps

- TensorRT engine building and profiling are implemented and guarded but not yet exercised on GPU hardware.
- The StyleGAN2-ADA generator has not been trained yet; preliminary counterfactuals use the placeholder generator.
- Real corpora (APTOS, EyePACS, Messidor-2, IDRiD) are not yet on disk; all functional checks use the synthetic corpus.

